

Lab 8: MLPClassifier Using Scikit-learn

Aim

To implement a **Multi-Layer Perceptron (MLP) classifier using Scikit-learn** for classification.

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create an MLPClassifier model with hidden layers.
5. Train the model using the training data.
6. Predict the classes for the test data.
7. Calculate and display the accuracy.

Program

```
# Lab 8: MLPClassifier Using Scikit-learn
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

```
# Load Iris dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# Create and train the MLP model
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(10, 10),  
    max_iter=1000,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred = model.predict(X_test)
```

```
print("Predictions:", y_pred)
```

```
print("Actual Values:", y_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Sample Output

```
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 1 0 2 2 2 2 2 0 0]
```

```
Actual Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 1 0 2 2 2 2 2 0 0]
```

```
Accuracy: 1.0
```

Result

Thus, the **MLPClassifier** was successfully implemented using **Scikit-learn** and used to classify the Iris dataset.